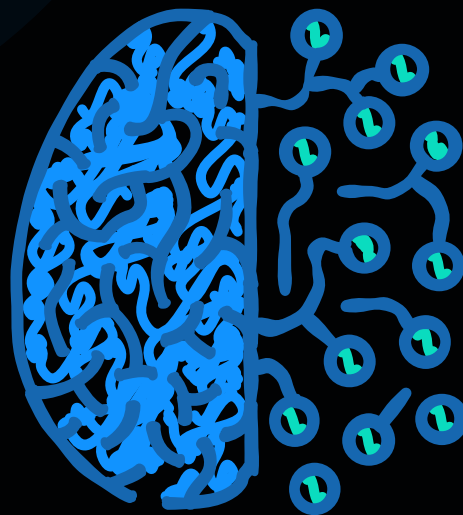


#4 Javascript Questions Asked in MAANG Companies

WHAT IS MEMOIZATION? IN JAVASCRIPT





WHAT IS **MEMOIZATION**?

- Memoization is a technique to optimize expensive function calls by caching their results.
- Every time a function runs with the same input — it returns the stored result instead of recalculating.





WHY IS **MEMOIZATION** IMPORTANT?

- Speeds up performance for recursive or heavy computations
- Great for interview questions like:
 - Fibonacci
 - Factorial
 - Dynamic programming
- Used in real-world apps: React re-renders, API calls caching, etc.





WITHOUT MEMOIZATION (INEFFICIENT)



```
function fib(n) {  
  if (n <= 1) return n;  
  return fib(n - 1) + fib(n - 2); // 🐌 Slow  
}  
console.log(fib(30)); // Takes time ⌚
```

- Calls repeat unnecessarily (re-computing the same inputs)



WITH MEMOIZATION (EFFICIENT)

```
function memoizedFib() {  
  const cache = {};  
  return function fib(n) {  
    if (n in cache) return cache[n];  
    if (n <= 1) return n;  
    cache[n] = fib(n - 1) + fib(n - 2);  
    return cache[n];  
  };  
}  
  
const fib = memoizedFib();  
console.log(fib(30)); // Much faster 🚀
```

- ✓ Caches results
- ✓ Avoids duplicate calculations
- ✓ Reduces time complexity bheading



MEMOIZATION TEMPLATE (REUSABLE)



```
function memoize(fn) {  
  const cache = {};  
  return function (...args) {  
    const key = JSON.stringify(args);  
    if (key in cache) return cache[key];  
    const result = fn(...args);  
    cache[key] = result;  
    return result;  
  };  
}
```

Use this to memoize any pure function 🧩



EXAMPLE USE CASE – FACTORIAL



```
const factorial = memoize(function(n) {  
  if (n === 0) return 1;  
  return n * factorial(n - 1);  
});  
console.log(factorial(5)); //  Cached
```



WHEN TO USE MEMOIZATION?

- ✓ Heavy recursive functions
- ✓ Expensive calculations
- ✓ Functions with same output for same input
- ⚠ Avoid if function has side effects or randomness



INTERVIEW TIP

“Memoization is like a memory booster for your code – if you’ve already done the hard work, don’t do it again!”

Mention this logic clearly during your interviews — especially in FAANG or system design questions.



Uzma Begum Shaik



@codewithuzma



Repost

